

New to machine learning? Watch a video course to get practical working knowledge of ML using web technologies [View series \(https://goo.gle/learn-webml\)](https://goo.gle/learn-webml)

Training models

This guide assumes you've already read the [models and layers \(/js/guide/models_and_layers\)](/js/guide/models_and_layers) guide.

In TensorFlow.js there are two ways to train a machine learning model:

1. using the Layers API with `LayersModel.fit()` (<https://js.tensorflow.org/api/latest/#tf.Model.fit>) or `LayersModel.fitDataset()` (<https://js.tensorflow.org/api/latest/#tf.Model.fitDataset>).
2. using the Core API with `Optimizer.minimize()` (<https://js.tensorflow.org/api/latest/#tf.train.Optimizer.minimize>).

First, we will look at the Layers API, which is a higher-level API for building and training models. Then, we will show how to train the same model using the Core API.

Introduction

A machine learning *model* is a function with learnable parameters that maps an input to a desired output. The optimal parameters are obtained by training the model on data.

Training involves several steps:

- Getting a [batch](https://developers.google.com/machine-learning/glossary/#batch) (<https://developers.google.com/machine-learning/glossary/#batch>) of data to the model.
- Asking the model to make a prediction.
- Comparing that prediction with the "true" value.
- Deciding how much to change each parameter so the model can make a better prediction in the future for that batch.

A well-trained model will provide an accurate mapping from the input to the desired output.

Model parameters

Let's define a simple 2-layer model using the Layers API:

```
const model = tf.sequential({
  layers: [
    tf.layers.dense({inputShape: [784], units: 32, activation: 'relu'}),
    tf.layers.dense({units: 10, activation: 'softmax'}),
  ]
});
```

Under the hood, models have parameters (often referred to as *weights*) that are learnable by training on data. Let's print the names of the weights associated with this model and their shapes:

```
model.weights.forEach(w => {
  console.log(w.name, w.shape);
});
```

We get the following output:

```
> dense_Dense1/kernel [784, 32]
> dense_Dense1/bias [32]
> dense_Dense2/kernel [32, 10]
> dense_Dense2/bias [10]
```

There are 4 weights in total, 2 per dense layer. This is expected since dense layers represent a function that maps the input tensor x to an output tensor y via the equation $y = Ax + b$ where A (the kernel) and b (the bias) are parameters of the dense layer.

NOTE: By default dense layers include a bias, but you can exclude it by specifying `{useBias: false}` in the options when creating a dense layer.

`model.summary()` is a useful method if you want to get an overview of your model and see the total number of parameters:

Layer (type)	Output shape	Param #
dense_Dense1 (Dense)	[null,32]	25120
dense_Dense2 (Dense)	[null,10]	330
Total params: 25450		
Trainable params: 25450		
Non-trainable params: 0		

Each weight in the model is backend by a **Variable**

(<https://js.tensorflow.org/api/0.14.2/#class:Variable>) object. In TensorFlow.js, a **Variable** is a floating-point **Tensor** with one additional method `assign()` used for updating its values. The Layers API automatically initializes the weights using best practices. For the sake of demonstration, we could overwrite the weights by calling `assign()` on the underlying variables:

```
model.weights.forEach(w => {  
  const newVals = tf.randomNormal(w.shape);  
  // w.val is an instance of tf.Variable  
  w.val.assign(newVals);  
});
```

Optimizer, loss and metric

Before you do any training, you need to decide on three things:

1. **An optimizer.** The job of the optimizer is to decide how much to change each parameter in the model, given the current model prediction. When using the Layers API, you can provide either a string identifier of an existing optimizer (such as 'sgd' or 'adam'), or an instance of the **Optimizer** (<https://js.tensorflow.org/api/latest/#Training-Optimizers>) class.
2. **A loss function.** An objective that the model will try to minimize. Its goal is to give a single number for "how wrong" the model's prediction was. The loss is computed on every batch of data so that the model can update its weights. When using the Layers API, you can provide either a string identifier of an existing loss function (such as 'categoricalCrossentropy'), or any function that takes a predicted and a true value and returns a loss. See a **list of available losses** (<https://js.tensorflow.org/api/latest/#Training-Losses>) in our API docs.
3. **List of metrics.** Similar to losses, metrics compute a single number, summarizing how well our model is doing. The metrics are usually computed on the whole data at the end of each epoch. At the very least, we want to monitor that our loss is going down over time. However, we often want a more human-friendly metric such as accuracy. When using the Layers API, you can provide either a string identifier of an existing metric (such as 'accuracy'), or any function that takes a predicted and a true value and returns a score. See a **list of available metrics** (<https://js.tensorflow.org/api/latest/#Metrics>) in our API docs.

When you've decided, compile a `LayersModel` by calling `model.compile()` with the provided options:

```
model.compile({  
  optimizer: 'sgd',  
  loss: 'categoricalCrossentropy',  
  metrics: ['accuracy']  
});
```

During compilation, the model will do some validation to make sure that the options you chose are compatible with each other.

Training

There are two ways to train a `LayersModel`:

- Using `model.fit()` and providing the data as one large tensor.
- Using `model.fitDataset()` and providing the data via a `Dataset` object.

`model.fit()`

If your dataset fits in main memory, and is available as a single tensor, you can train a model by calling the `fit()` method:

```
// Generate dummy data.  
const data = tf.randomNormal([100, 784]);  
const labels = tf.randomUniform([100, 10]);  
  
function onBatchEnd(batch, logs) {  
  console.log('Accuracy', logs.acc);  
}  
  
// Train for 5 epochs with batch size of 32.  
model.fit(data, labels, {  
  epochs: 5,  
  batchSize: 32,  
  callbacks: {onBatchEnd}  
}).then(info => {  
  console.log('Final accuracy', info.history.acc);  
});
```

Under the hood, `model.fit()` can do a lot for us:

- Splits the data into a train and validation set, and uses the validation set to measure progress during training.
- Shuffles the data but only after the split. To be safe, you should pre-shuffle the data before passing it to `fit()`.
- Splits the large data tensor into smaller tensors of size `batchSize`.
- Calls `optimizer.minimize()` while computing the loss of the model with respect to the batch of data.
- It can notify you on the start and end of each epoch or batch. In our case, we are notified at the end of every batch using the `callbacks.onBatchEndoption`. Other options include: `onTrainBegin`, `onTrainEnd`, `onEpochBegin`, `onEpochEnd` and `onBatchBegin`.
- It yields to the main thread to ensure that tasks queued in the JS event loop can be handled in a timely manner.

For more info, see the [documentation](https://js.tensorflow.org/api/latest/#tf.Sequential.fit) (<https://js.tensorflow.org/api/latest/#tf.Sequential.fit>) of `fit()`. Note that if you choose to use the Core API, you'll have to implement this logic yourself.

`model.fitDataset()`

If your data doesn't fit entirely in memory, or is being streamed, you can train a model by calling `fitDataset()`, which takes a `Dataset` object. Here is the same training code but with a dataset that wraps a generator function:

```
function* data() {
  for (let i = 0; i < 100; i++) {
    // Generate one sample at a time.
    yield tf.randomNormal([784]);
  }
}

function* labels() {
  for (let i = 0; i < 100; i++) {
    // Generate one sample at a time.
    yield tf.randomUniform([10]);
  }
}

const xs = tf.data.generator(data);
const ys = tf.data.generator(labels);
// We zip the data and labels together, shuffle and batch 32 samples at a time.
const ds = tf.data.zip({xs, ys}).shuffle(100 /* bufferSize */).batch(32);

// Train the model for 5 epochs.
model.fitDataset(ds, {epochs: 5}).then(info => {
```

```
    console.log('Accuracy', info.history.acc);  
  });
```

For more info about datasets, see the [documentation](#)

(<https://js.tensorflow.org/api/latest/#tf.Model.fitDataset>) of `model.fitDataset()`.

Predicting new data

Once the model has been trained, you can call `model.predict()` to make predictions on unseen data:

```
// Predict 3 random samples.  
const prediction = model.predict(tf.randomNormal([3, 784]));  
prediction.print();
```

Note: As we mentioned in the [Models and Layers](#) (/js/guide/models_and_layers) guide, the `LayersModel` expects the outermost dimension of the input to be the batch size. In the example above, the batch size is 3.

Core API

Earlier, we mentioned that there are two ways to train a machine learning model in TensorFlow.js.

The general rule of thumb is to try to use the Layers API first, since it is modeled after the well-adopted Keras API. The Layers API also offers various off-the-shelf solutions such as weight initialization, model serialization, monitoring training, portability, and safety checking.

You may want to use the Core API whenever:

- You need maximum flexibility or control.
- And you don't need serialization, or can implement your own serialization logic.

For more information about this API, read the "Core API" section in the [Models and Layers](#) (/js/guide/models_and_layers) guide.

The same model as above written using the Core API looks like this:

```
// The weights and biases for the two dense layers.  
const w1 = tf.variable(tf.randomNormal([784, 32]));
```

```
const b1 = tf.variable(tf.randomNormal([32]));
const w2 = tf.variable(tf.randomNormal([32, 10]));
const b2 = tf.variable(tf.randomNormal([10]));

function model(x) {
  return x.matMul(w1).add(b1).relu().matMul(w2).add(b2);
}
```

In addition to the Layers API, the Data API also works seamlessly with the Core API. Let's reuse the dataset that we defined earlier in the `model.fitDataset()` (`#model.fitDataset()`) section, which does shuffling and batching for us:

```
const xs = tf.data.generator(data);
const ys = tf.data.generator(labels);
// Zip the data and labels together, shuffle and batch 32 samples at a time.
const ds = tf.data.zip({xs, ys}).shuffle(100 /* bufferSize */).batch(32);
```

Let's train the model:

```
const optimizer = tf.train.sgd(0.1 /* learningRate */);
// Train for 5 epochs.
for (let epoch = 0; epoch < 5; epoch++) {
  await ds.forEachAsync(({xs, ys}) => {
    optimizer.minimize(() => {
      const predYs = model(xs);
      const loss = tf.losses.softmaxCrossEntropy(ys, predYs);
      loss.data().then(l => console.log('Loss', l));
      return loss;
    });
  });
  console.log('Epoch', epoch);
}
```

The code above is the standard recipe when training a model with the Core API:

- Loop over the number of epochs.
- Inside each epoch, loop over your batches of data. When using a `Dataset`, `dataset.forEachAsync()` (<https://js.tensorflow.org/api/0.15.1/#tf.data.Dataset.forEachAsync>) is a convenient way to loop over your batches.
- For each batch, call `optimizer.minimize(f)` (<https://js.tensorflow.org/api/latest/#tf.train.Optimizer.minimize>), which executes `f` and minimizes its output by computing gradients with respect to the four variables we defined earlier.

- `f` computes the loss. It calls one of the predefined loss functions using the prediction of the model and the true value.

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2025-05-13 UTC.